09 排序

- 基本排序方法
- 归并排序
- 快速排序
- 堆排序
- 希尔排序
- 基数排序

09 排序

- **基本排序方法**
- 归并排序
- **快速排序**
- **堆排序**
- **希尔排序**
- **基数排序**

排序

- **排序(Sorting)是将一组数据元素(或记录)的任意序列重新排列成一个按关键码有序的序列**
  - **升序：关键码从小到大排序**
  - **降序：关键码从大到小排序**

- **排序的稳定性：若序列中关键码相等的数据元素经过某种排序方法排序后，仍能保持它们在排序前的相对顺序，则该排序方法是稳定的；否则该排序方法是不稳定的**
  - **待排序列：34, 12, 34’, 08, 96**
  - **稳定排序结果：08, 12, 34, 34’, 96**
  - **不稳定排序结果：08, 12, 34’, 34, 96**

排序的分类

- 根据内外存使用情况
  - 内部排序：待排序数据均在内存中进行排序(本章所有算法均为内部排序算法)
  - 外部排序：部分数据在外存，部分数据在内存进行调整，逐步完成全部数据的排序(本章不涉及)
- 根据排序实现手段
  - 基于“比较-交换”的排序，通过对关键码的比较，交换数据在序列中的位置：插入排序、冒泡排序、选择排序、归并排序、快速排序、堆排序、希尔排序
  - 基于“分配-收集”的排序：基数排序
- 根据算法逻辑的难易程度
  - 基本排序：直接插入排序、冒泡排序、直接选择排序
  - 高级排序：归并排序、快速排序、堆排序、希尔排序、基数排序

直接插入排序(Insertion Sort)

- 基本思路：将一个数据插入到已排好序的序列中，形成一个新的、数据数增1的有序序列

- 设$a_0, a_1, \ldots, a_{i-1}$已排好序($a_0 \le a_1 \le \ldots \le a_{i-1}$)，对于$t = a_i$，依次与$a_{i-1}, a_{i-2}, \ldots$进行比较，将比t大的数据右移一位，直到发现某个$a_j \le t$，则$a_{j+1} = t$，若不存在$a_j$，则$a_0 = t$

直接插入排序示例

45	34	78	12	34’	32	29	64
45							
34	45	78					
12	34	45	78				
12	34	34’	45	78			

稳定

直接插入排序实现

```
template <class Item>
void InsertionSort(Item a[], int left, int right) {
    int i, j;
    Item t;
    for (i = left + 1; i <= right; i++) {
        for (j = i - 1, t = a[i]; j >= left && t < a[j]; j--)
            a[j+1] = a[j];
        a[j+1] = t;
    }
}
```

直接插入排序性能分析

- 时间代价
  - **最好情况(排序)：O(n)**
    - 比较n-1次，移动0次
  - **最坏情况(逆序)： $O(n^2)$**
    - 比较$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$次，移动$\sum_{i=1}^{n-1}(i+1) = \frac{(n+2)(n-1)}{2}$次
    - **平均情况：$O(n^2)$**
- **空间代价：O(1)**

自身插入也算

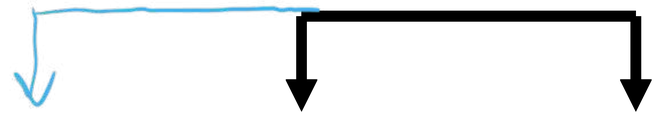
- **稳定排序**
- **适用数据量少的场合**

冒泡排序(Bubble Sort)

- **基本思路：依次比较相邻两个数据的大小，如果顺序不对，则将两者交换，重复这样的操作直到整个序列有序**
- **实际是通过比较与交换使得待排序列一个最值元素“上浮”到序列一端，然后缩小排序范围**

- **设待排序的数据为$a_0, a_1, a_2, \ldots, a_{n-1}$，初始时排序范围是从$a_0$至$a_{n-1}$。可以自左向右，也可自右向左冒泡**
  - **例如，自右向左冒泡时，对相邻两数据进行比较，不符合顺序的相邻数据交换位置；当比较完当前排序范围后，关键码最小的数据被移动到序列的a_0位置，故a_0无需再参加下一轮比较，即下一轮比较的范围为a_1至a_{n-1}**

冒泡排序示例



45	34	78	12	34’	32	29	64
12	45	34	78	29	34’	32	64
12	29	45	34	78	32	34’	64

冒泡排序实现

```
template <class Item>
void BubbleSort(Item a[], int left, int right) {
    int exchange = 1;
    for (int i = left; i < right && exchange; i++)
        // 自右向左冒泡
        exchange = 0;
        for (int j = right - 1; j >= i; j--)
            if (a[j] > a[j+1]) {
                swap(a[j], a[j+1]);
                exchange = 1;
            }
}
```

冒泡排序性能分析

4 3 2 1
1 4 3 2

- 时间代价
  - **最好情况(排序)：O(n)**
    - 比较n-1次，移动0次
  - **最坏情况(逆序)： $O(n^2)$**
    - 比较$\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}$次，移动$3\sum_{i=1}^{n-1}(n-i)=\frac{3n(n-1)}{2}$次
    - **平均情况：$O(n^2)$**
- **空间代价：O(1)**

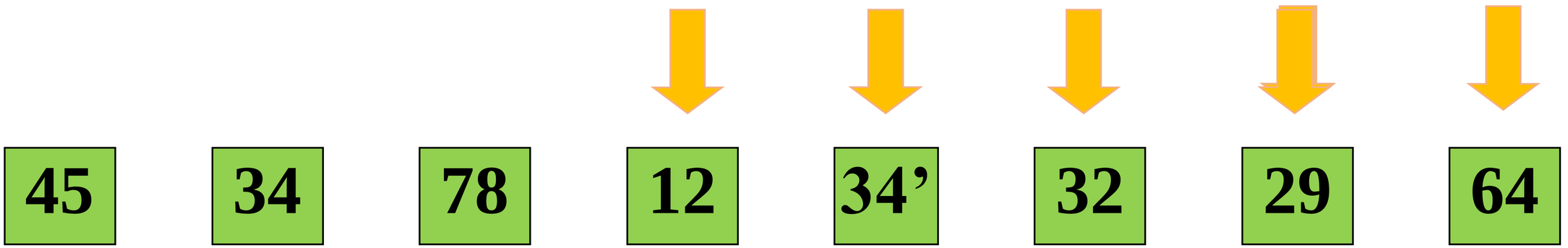
中间变量

- **稳定排序**

直接选择排序(Selection Sort)

- **基本思路：首先选出关键码最小的数据，将它与第一个数据交换位置；然后选出关键码次小的数据，将其与第二个数据交换位置；...；直到整个序列完成排序**
- **设待排序的数据为$a_0, a_1, a_2, \ldots, a_{n-1}$，　依次对i = 0, 1, …, n-2分别执行如下的选择步骤**
  - **在$a_i, a_{i+1}, \ldots, a_{n-1}$中选择一个关键码最小的数据$a_k$**
  - **将a_k与a_i交换**

直接选择排序示例



12

45

复杂度大

不稳定

直接选择排序实现

```
template <class Item>
void SelectionSort(Item a[], int left, int right) {
    int i, j, min;
    for (i = left; i < right; i++) {
        for (min = i, j = i + 1; j <= right; j++)
            if (a[j] < a[min])
                min = j;
        swap(a[i], a[min]);
    }
}
```

直接选择排序性能分析

- 时间代价
  - **最好情况(排序)：$O(n^2)$**
    - 比较$\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}$次，移动0次
  - **最坏情况：$O(n^2)$**
    - 比较$\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}$次，移动$3(n-1)$ 次
  - **平均情况：$O(n^2)$**
    - 运行时间与数据初始顺序关系很小
    - 交换次数很少
- **空间代价：$O(1)$**
- **不稳定排序**

4 3 2 1

1 3 2 4

比较如3交换 1 2 3 4

09 排序

- 基本排序方法
- 归并排序
- 快速排序
- 堆排序
- 希尔排序
- 基数排序

归并排序(Merge Sort)

- **基本思路：将两个或两个以上的有序序列合并为一个新的有序序列**
  - **2路归并：比较两个已排序的序列各自的最小数据，选出较小的数据放入新序列中，循环直到两个序列中所有数据均在新序列中**
  - **多路归并：选择合并的序列多于2**

- 归并的次序
  - **自顶向下**
  - **自底向上**

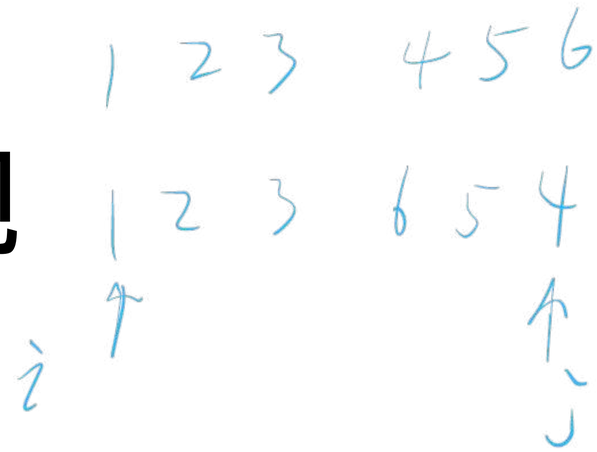
自顶向下归并排序示例


(25 34 45 32 78 12 34’ 64)
先划分
(25 34 45 32)(78 12 34’ 64)
(25 34)(45 32)(78 12)(34’ 64)
(25)(34)(45)(32)(78)(12)(34’)(64)
再归并
(25 34)(32 45)(12 78)(34’ 64)
(25 32 34 45)(12 34’ 64 78)
(12 25 32 34 34’ 45 64 78)

自顶向下归并排序实现

```
template <class Item>
void MergeSort(Item a[], int left, int right) {
    // 区间内只有一个数据，返回
    if (right <= left) return;
    int m = (right + left) / 2;
    // 对区间前半段进行递归归并排序
    MergeSort(a, left, m);
    // 对区间后半段进行递归归并排序
    MergeSort(a, m + 1, right);
    // 对两个排好序的序列进行归并
    merge(a, left, m, right);
}
```

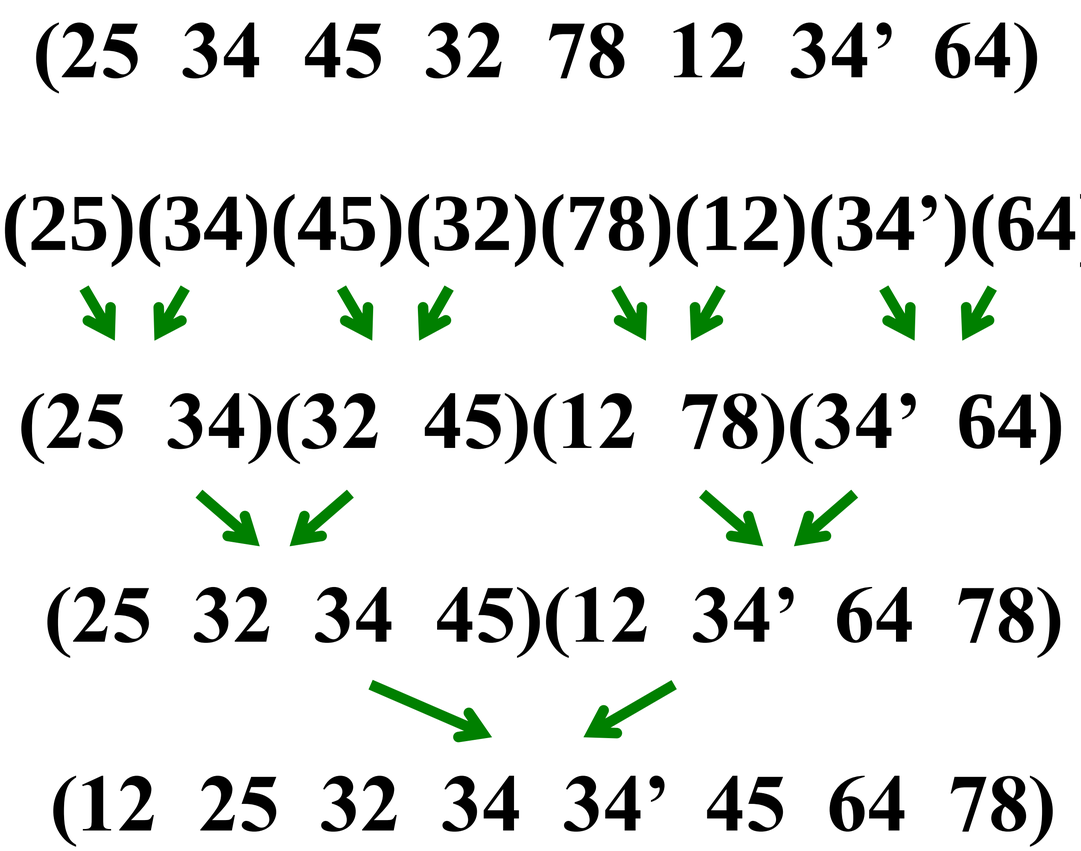
自顶向下归并排序实现




```
template <class Item>
void merge(Item a[], int left, int m, int right) {
    // 初始时a[left]~a[m]，a[m+1]~a[right]各自为升序
    int i, j;  static Item aux[N];
    // 重新生成序列，前半部分升序，后半部分降序
    for (i = m+1; i > left; i--)  aux[i-1] = a[i-1];
    for (j = m; j < right; j++)  aux[m+right-j] = a[j+1];
    // 将aux数组中的两个序列归并到a数组
    for (int k = left; k <= right; k++)
        if (aux[j] < aux[i])  a[k] = aux[j--];
        else  a[k] = aux[i++];
}
```

中间为啥会无需判断其中有一数组已排完

自底向上归并排序示例


(25 34 45 32 78 12 34’ 64)
(25)(34)(45)(32)(78)(12)(34’)(64)
(25 34)(32 45)(12 78)(34’ 64)
(25 32 34 45)(12 34’ 64 78)
(12 25 32 34 34’ 45 64 78)

自底向上归并排序实现

```
inline int min(int A, int B) { return A <= B ? A : B; }
template <class Item>
void MergeSort(Item a[], int left, int right) {
   // 归并：首先按步长为1进行归并，然后按步长×2进行归并，
   // ...，直到步长大于当前序列长度
   for (int m = 1; m <= right - left; m = m + m)
      for (int i = left; i <= right - m; i += m + m)
         // 右边界不要超出序列范围
         merge(a, i, i + m - 1, min(i + m + m - 1, right));
}
```

m = m * 2

(i + m - 1) + m

左　中　右

归并排序性能分析

- 时间代价：$O(nlog_2n)$
  - $T(n) = c*n + 2*T(n/2)$
    $= c*n + 2*(c*n/2 + 2*T(n/2^2))$
    $= 2*c*n + 2^2*T(n/2^2)$
    $= k*c*n + 2^k*T(n/2^k)$
    $= O(nlog_2n)$
  - 运行时间与数据初始顺序关系很小
- 空间代价：$O(n)$

- 稳定排序

$\log_2 n$ 次
每次遍历一趟 n

$2^k = n$
$k = \log_2 n$

09 排序

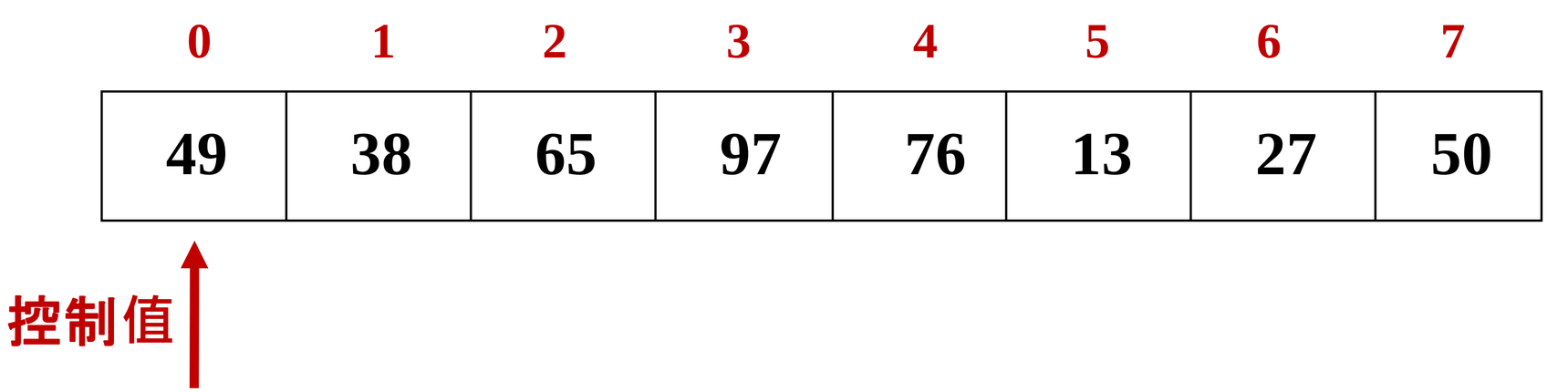
- 基本排序方法
- 归并排序
- **快速排序**
- 堆排序
- 希尔排序
- 基数排序

快速排序(Quick Sort)

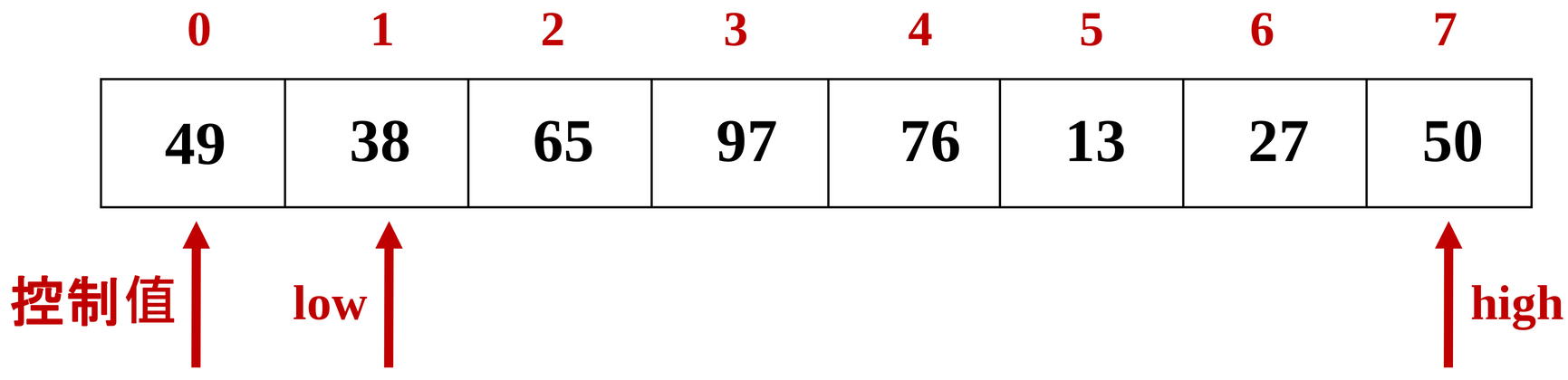
- 平均性能最好的排序算法
- 基本思路：
  - 取待排序数据序列中的某个数据作为控制值
  - 放置控制值到序列的合适位置，使得控制值左边所有数据小于等于控制值，右边所有数据大于控制值
  - 对控制值左、右两边的序列分别进行以上操作

- 快速排序的关键操作是**划分**(partition)，其主要作用是确定控制值的位置，从而缩小待排序列数据的交换范围，使其只能在控制值左(右)边序列范围内移动

划分过程示例



(1) t = a[low]，即控制值为当前序列的第一个数据，为了减少数据的移动，将作为控制值的数据暂存到t中，最后再放入最终位置



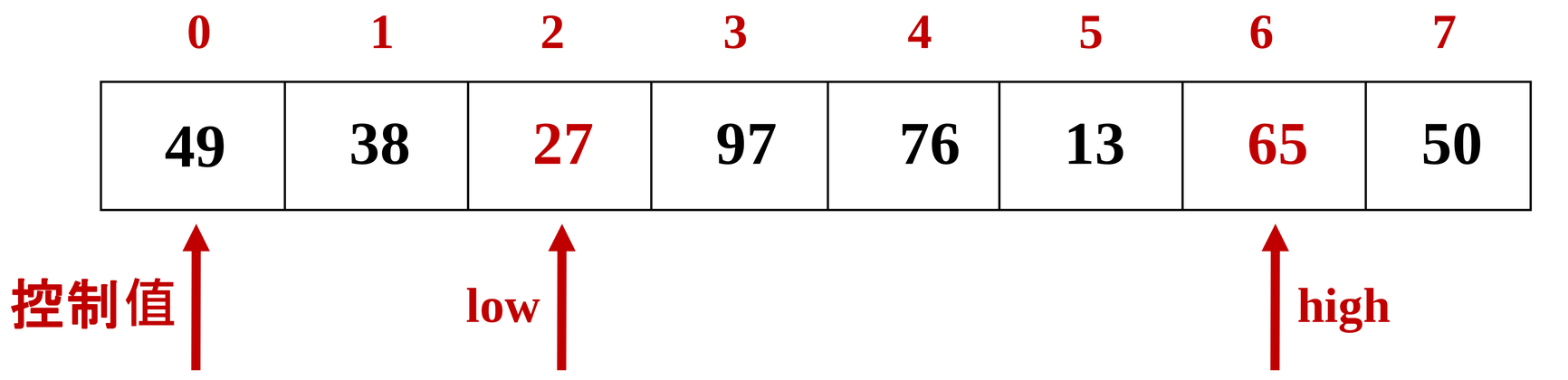
(2) low指向待划分区域首元素，high指向待划分区域尾元素

划分过程示例



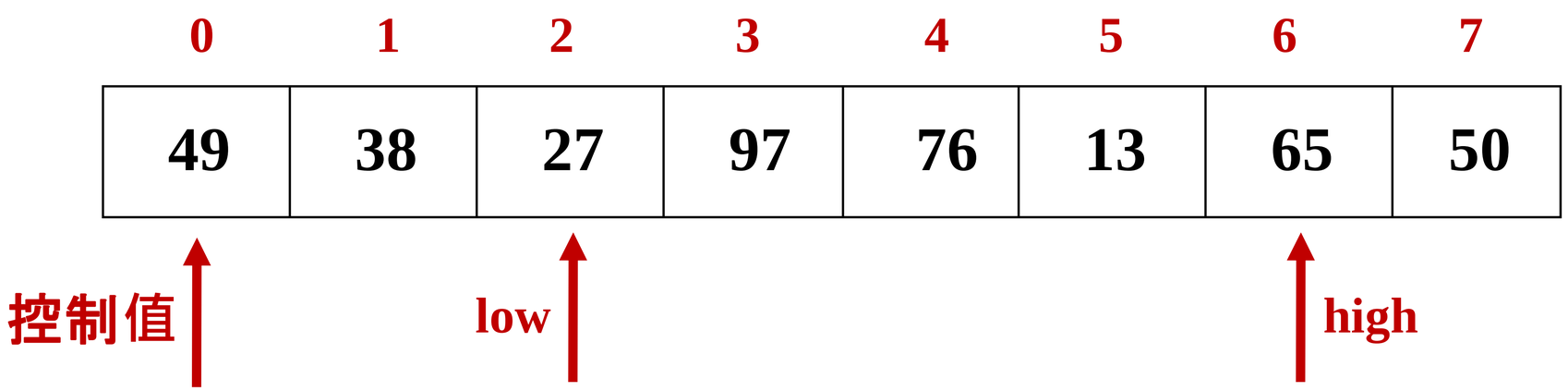
找到位置及其他元素

(3) high从后往前移动直到a[high] <= t,　low从前往后移动直到a[low] >= t

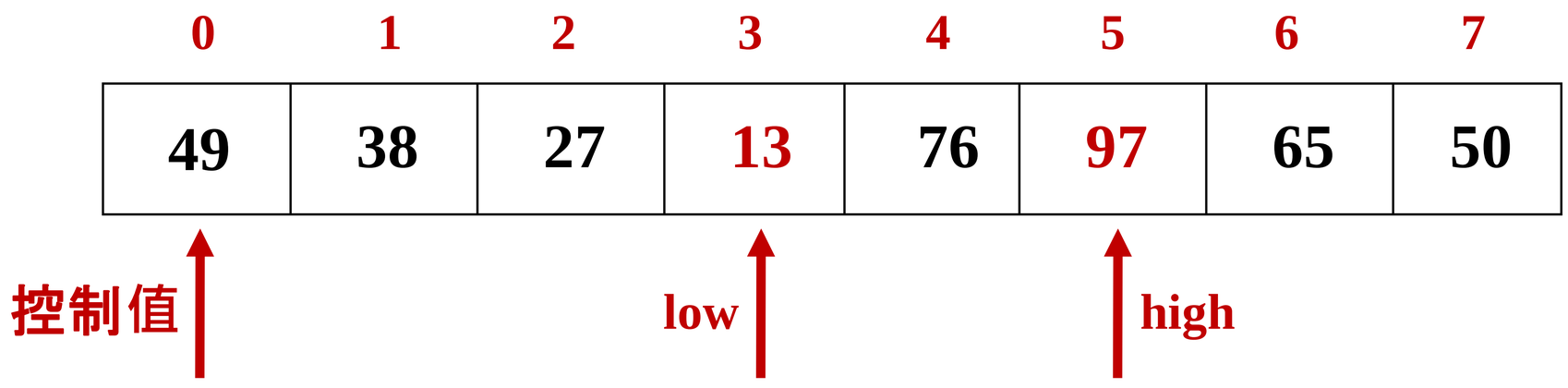


(4) 交换a[low]和a[high]

划分过程示例

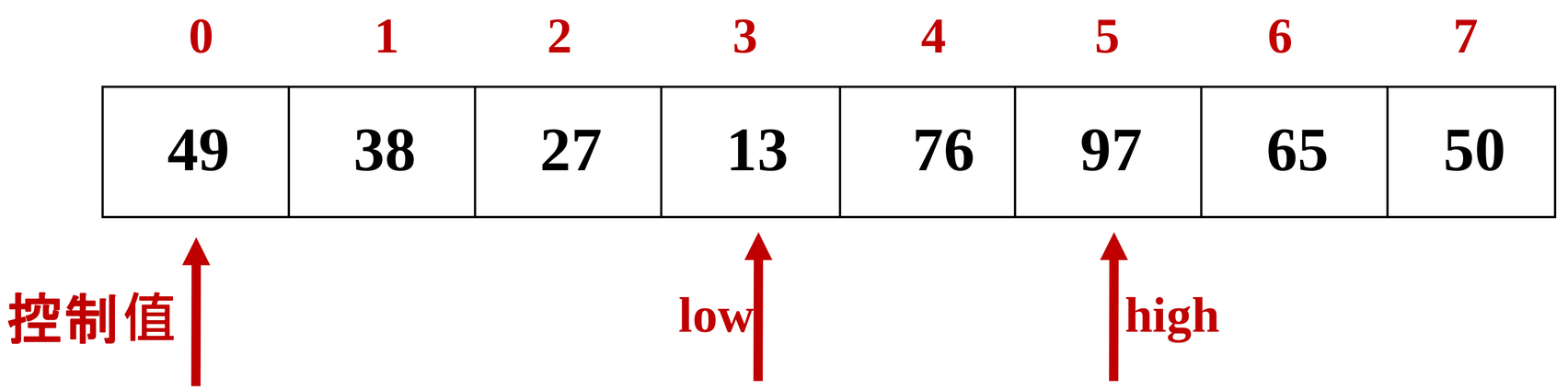


(5) high从后往前移动直到a[high] <= t,　low从前往后移动直到a[low] >= t

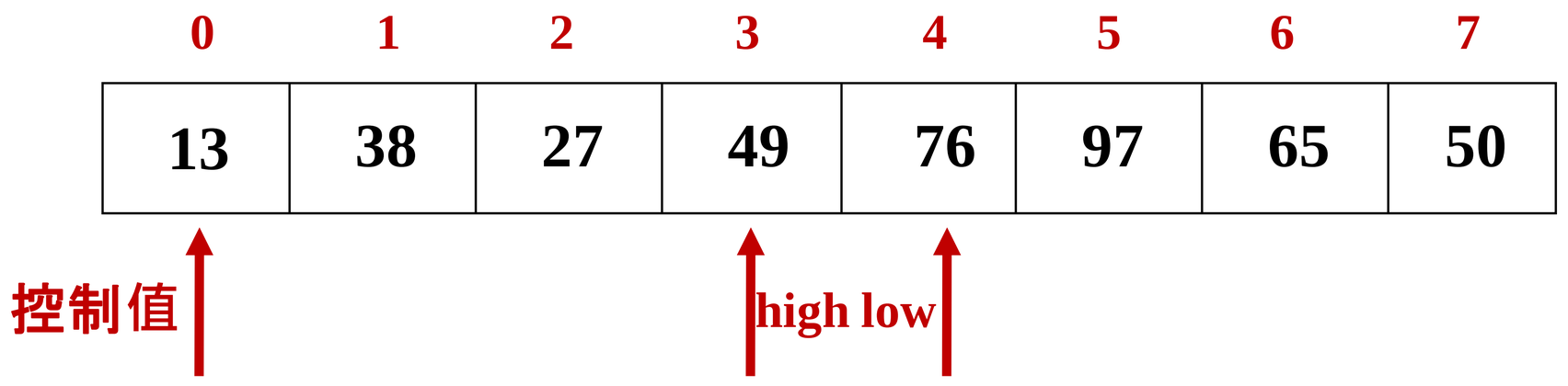


(6) 交换a[low]和a[high]

划分过程示例



(7) high从后往前移动直到a[high] <= t,　low从前往后移动直到a[low] >= t



(8) low >= high,　交换a[high]和t

划分实现

```
template <class Item>
int partition(Item a[], int left, int right) {
    Item t = a[left];
    int i = left, j = right + 1;
    for (;;) {
        while (a[++i] < t);  // 从前往后
        while (t < a[--j]) if (j == left) break;  // 从后往前
        if (i >= j)  break;
        swap(a[i], a[j]);
    }
    swap(a[j], a[left]);
    return j;
}
```

快速排序实现

```
template <class Item>
void QuickSort(Item a[], int left, int right) {
    int i;
    if (right <= left)  return;
    i = partition(a, left, right);
    QuickSort(a, left, i-1);
    QuickSort(a, i+1, right);
}
```

快速排序性能分析

- 时间代价
  - **最好情况：$O(nlog_2n)$**
    - $T(n) = c*n + 2*T(n/2) = O(nlog_2n)$
  - **最坏情况(排序或逆序)：$O(n^2)$**
    - $T(n) = c*n + T(0) + T(n-1) = O(n^2)$
  - **平均情况：$O(nlog_2n)$**
    - $T(n) = c*n + 1/n*\sum_{k=0}^{n-1}(T(k)+T(n-1-k)) = c*n + 2/n*\sum_{k=0}^{n-1}T(k) = O(nlog_2n)$
- **空间代价(递归压栈)：O(n)**
- **不稳定排序**

快速排序的改进：栈的使用

- 程序递归需要系统栈空间
  - 最坏情况需要O(n)
  - 若n很大，则系统栈会因为缺少内存空间而非正常结束
- 改进：优先处理短序列，则栈的深度最多为$\log_2 n$

```
template <class Item>
void QuickSort(Item a[], int left, int right) {
    int i;
    if (right <= left)  return;
    i = partition(a, left, right);
    if (i - left > right - i) {
       QuickSort(a, i+1, right); QuickSort(a, left, i-1);
    } else {
       QuickSort(a, left, i-1); QuickSort(a, i+1, right);
    }
}
```

快速排序的改进：非递归

```
inline void push_interval(Stack <int> &s, int a, int b) { s.push(b);  s.push(a); }
template <class Item>
void QuickSort(Item a[], int left, int right) {
    Stack<int> s;  int i;
    push_interval(s, left, right);  // 将初始区间压栈
    while(!s.empty()) {  // 栈空说明已经排好序
        s.pop(&left);  s.pop(&right);
        if (right <= left) continue;  // 若左右边界重合，则结束当前循环
        i = partition(a, left, right);
        if (i - left > right - i) {
            push_interval(s, left, i-1);  push_interval(s, i+1, right);
        } else {
            push_interval(s, i+1, right);  push_interval(s, left, i-1);
        }
    }
}
```

快速排序的改进：控制值的选择

- 当控制值不能将序列一分为二，则会退化成$O(n^2)$
- 改进：
  - 随机选择降低最坏发生的概率(最坏情况还是会发生)
  - 从序列中选择三个数据(例如，left, mid, right)，然后选择三个数据中的中间数据(并交换到right的位置)

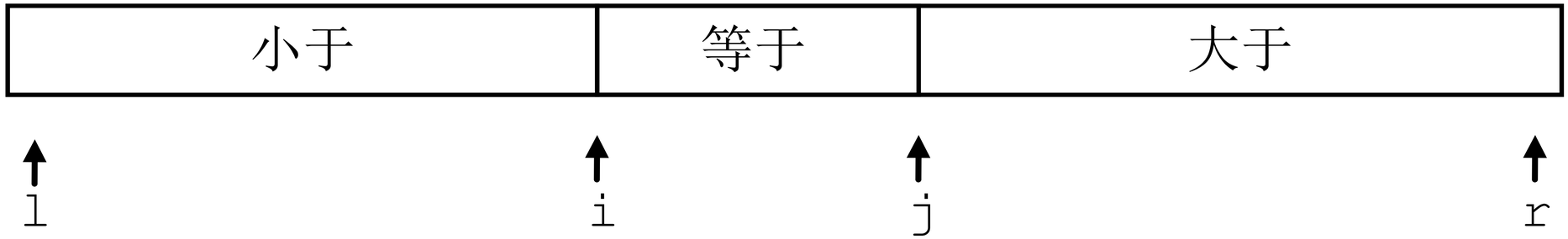
快速排序的改进：短子序列

- 对于短序列，**快速排序效率不如一些**简单的排序方法
  - **插入排序在**这种情况表现更好

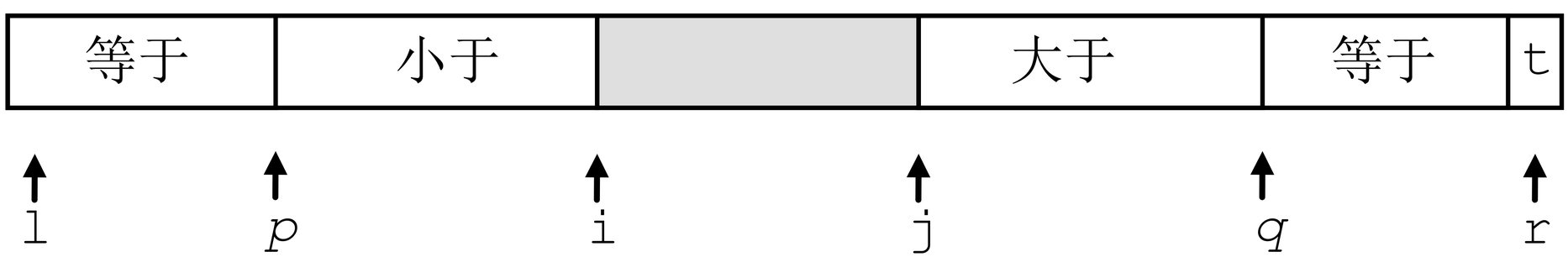
- 设M为切换到插入排序时短子序列长度阈值
  - **研究表明，M取值在5~25时，效率比M=1提高10%**
  - **经验表明，M取值在8~10时效率会更高**

快速排序的改进：重复值的处理

- 当存在大量重复结点时，快速排序算法会退化
- 改进：三路划分(小于、等于、大于)



- 扫描左子序列，将与控制值相等的数据放在该子序列的最左端
- 扫描右子序列，将与控制值相等的数据放在该子序列的最右端
- 当i和j相遇，循环将等于控制值的数据交换到序列中央



快速排序的改进：重复值的处理

```
template <class Item>
void QuickSort(Item a[], int left, int right) {
    int k;  Item v = a[right];  if (right <= left)  return;
    int  i = left-1, j = right, p = left-1, q = right;
    for(;;) {
        while(a[++i] < v);  while (v < a[--j])  if (j == left) break;
        if (i >= j) break;  swap(a[i], a[j]);
        if (a[i] == v) { p++;  swap(a[p], a[i]); }
        if (v == a[j]) { q--;  swap(a[q], a[j]); }
    }
    swap(a[i], a[right]);  j = i-1; i = i+1;
    for (k = left; k <= p; k++, j--) swap(a[k], a[j]);  // 换到序列正中
    for (k = right-1; k >= q; k--, i++) swap(a[k], a[i]);  // 换到序列正中
    QuickSort(a, left, j);  QuickSort(a, i, right);
}
```

09 排序

- 基本排序方法
- 归并排序
- 快速排序
- **堆排序**
- 希尔排序
- 基数排序

堆排序(Heap Sort)

- 建立最大堆，堆顶数据与堆底数据互换，自上而下调整堆，重复执行上述步骤直至整个序列完成排序
- 时间复杂度为$O(nlog_2n)$
- 空间复杂度为$O(1)$
- 不稳定的排序

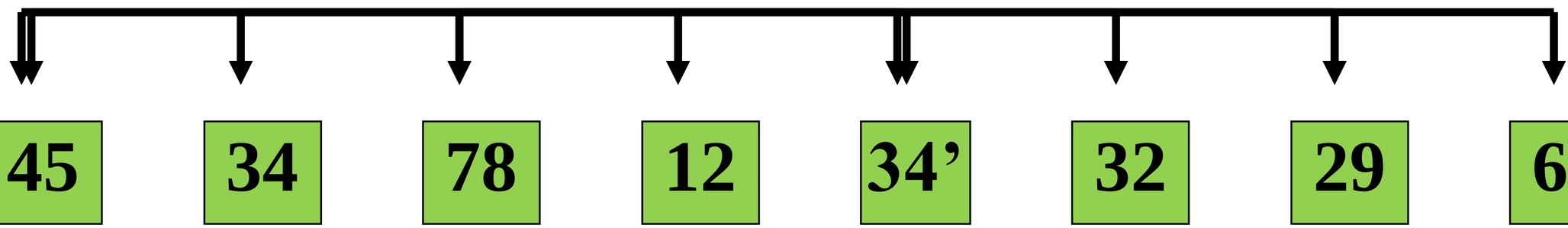
09 排序

- 基本排序方法
- 归并排序
- 快速排序
- 堆排序
- **希尔排序**
- 基数排序

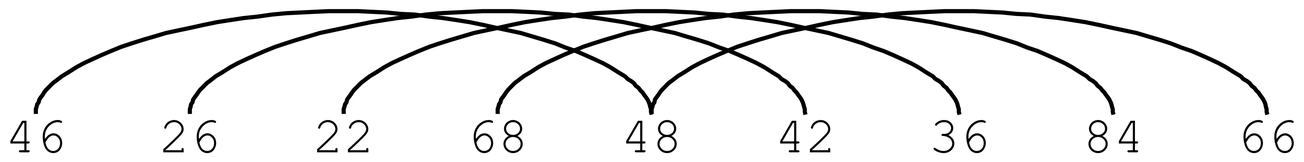
希尔排序(Shell Sort)

- 插入排序的两个特点
  - 在最好情况下(序列本身是有序)，时间复杂度为O(n)
  - 对于短序列，插入排序很高效
- 希尔排序有效地利用了插入排序的这两个特点
  - 给定一组严格递减的正整数增量$d_0, d_1, \ldots, d_{t-1}$，且取$d_{t-1} = 1$
  - 依次将待排序序列切分为d_i组，每组中相邻数据的下标相差d_i，对每组数据进行插入排序
  - 即逐渐扩大子序列的规模，而减少子序列个数，使得待排序序列逐渐处于更有序的状态

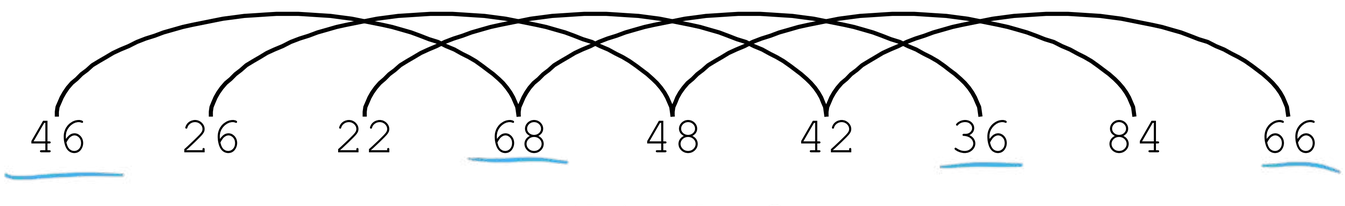
希尔排序示例



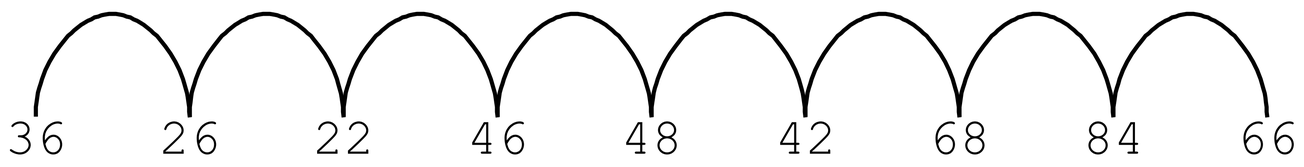
希尔排序示例



(a) $d_0=4$



(b) $d_1=3$



(c) $d_0=1$

22 26 36 42 46 48 66 68 84

(d) 排序完成

希尔排序实现

```
const int d[] = {4, 3, 1};
const int t = 3;
template <class Item>
void ShellSort(Item a[], int left, int right) {
    int h, m, i, j;
    Item v;
    for (m = 0; m < t; m++)  // 总计包括t种步长，循环t次
        // 依次对每个步长所确定的子序列进行插入排序
        for (h = d[m], i = left + h; i <= right; i++) {
            for (j = i, v = a[i]; j >=  left + h && v < a[j-h]; j -= h)
                a[j] = a[j-h];
            a[j] = v;
        }
}
```

希尔排序性能分析

- 时间代价依赖于增量序列，没有确切结论
  - **增量序列满足$n/2^i$，最坏情况下为$O(n^2)$**
  - **增量序列满足2^k-1，最坏情况下为$O(n^{1.5})$**
- **空**间代价：**O(1)**

- **不稳定排序**

09 排序

- 基本排序方法
- 归并排序
- 快速排序
- 堆排序
- 希尔排序
- **基数排序**

元组的比较

- 两个d元组$(x_0, x_1, \ldots, x_{d-1})$和$(y_0, y_1, \ldots, y_{d-1})$，当且仅当$x_0 = y_0$ && $x_1 = y_1$ && … && $x_i = y_i$ $(0 \le i < d-1)$时
  - 如果$x_{i+1} < y_{i+1}$，则$(x_0, x_1, \ldots, x_{d-1}) < (y_0, y_1, \ldots, y_{d-1})$
  - 如果$x_{i+1} > y_{i+1}$，则$(x_0, x_1, \ldots, x_{d-1}) > (y_0, y_1, \ldots, y_{d-1})$
- 从高位(最左位)起第一个不相等的位决定两个元组的大小



基数排序(Radix Sort)

- 前述的排序都是基于“比较-交换”的排序
- 基数排序是基于“分配-收集”的排序
  - 不需要进行记录之间的两两比较，而需要知道数据的每位信息
- 待排序数据序列$v_0, v_1, \ldots, v_{n-1}$，排序若满足：
  - 数据v_i的关键码是由d元组$k_i = (k_i^0, k_i^1, \ldots, k_i^{d-1})$组成，$k_i^0$、$k_i^{d-1}$分别为关键码的最高位、最低位
  - 任意v_i和v_j $(i < j)$都有$(k_i^0, k_i^1, \ldots, k_i^{d-1}) \le (k_j^0, k_j^1, \ldots, k_j^{d-1})$
- 则称这样的排序为基数排序
- 按照元组元素从高位还是低位开始排序分两种
  - 最高位优先(most significant digit first，缩写MSD)
  - 最低位优先(least significant digit first，缩写LSD)

MSD基数排序

- 先对最高位k^0排序，分为若干序列，每个序列的k^0相同
- 然后对每个序列按次高位k^1排序，分成更小的序列，每个序列的k^0和k^1相同
- 依次重复，直到对k^{d-1}排序后，分成最小的序列，每个序列内含有相同的$(k^0, k^1, \ldots, k^{d-1})$
- 最后将所有的序列依次连接在一起，成为一个有序序列
- 这是一个分配、分配、…、分配、收集的过程

基于数组的MSD基数排序实现

```
#define bin(A) left + count[A]  // bin(A)计算A桶的地址
const int radix = 256;  // 选择一个字节作为基数
const int M = 3;  // 若待排序数据个数小于M，用插入排序
const int maxN = 1000;
const int bytes = 4;  // 每个数据4个字节
template <typename Item>
inline int getDigit(Item A, int B) {  // 取A的第B个字节
    A = A>>((bytes-B-1)*8);
    return A%radix;
}
```

基于数组的MSD基数排序实现

```
template <typename Item>
void RadixSort_MSD(Item a[], int left, int right, int d) {  // 第d位排序
    int i, j, count[radix+1];  static Item aux[maxN];  if (d>bytes) return;
    if (right - left + 1 <= M) { InsertionSort(a, left, right);  return; } //太小，直接插入排序
    for (j = 0; j <= radix; j++) count[j] = 0;  // 初始化各桶数据个数
    // 根据数据的第d位将数据放入对应的桶内
    for (i = left; i <= right; i++) count[getDigit(a[i], d)+1]++;
    for (j = 1; j <= radix; j++) count[j] += count[j-1];  // 各桶起点位置
    // 将各桶数据按桶的顺序放置在辅助数组aux中
    for (i = left; i <= right; i++) aux[count[getDigit(a[i], d)]++] = a[i];
    for (i = left; i <= right; i++) a[i] = aux[i-left];  // 将aux数组写回原数组
    // 对各桶递归
    RadixSort_MSD(a, left, bin(0)-1, d+1);
    for (j = 0; j < radix-1; j++) RadixSort_MSD(a, bin(j), bin(j+1)-1, d+1);
}
```

LSD基数排序

- 先对最低位k^{d-1}排序，分为若干序列，再按k^{d-1}的值从小到大依次将各个序列收集起来
- 再对次低位k^{d-2}排序，分为若干序列，再按k^{d-2}的值从小到大依次将各个序列收集起来
- 依次重复，直至对最高位k^0排序，分为若干序列，再按k^0的值从小到大依次将各个序列收集起来，就是有序的
- 这是一个分配、收集；分配、收集；…；分配、收集的过程，相对于MSD比较简单，是计算机常用的方式

LSD基数排序实现

- 基于顺序存储结构(数组)
  - 速度快，通用
- 基于链式存储结构
  - 避免空间浪费

基于数组的LSD基数排序示例

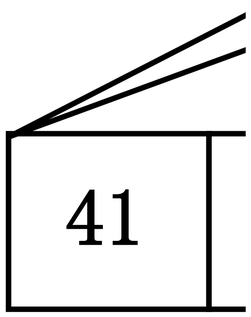
初始数组内容：　97　53　88　59　26　41　88’　31　22

个位：　0　1　2　3　4　5　6　7　8　9

第一趟：count　**0**　2　1　1　**0**　**0**　1　1　2　1

0　1　2　3　4　5　6　7　8　9

按 count 分配桶：　**0**　2　3　4　**4**　**4**　5　6　**8**　**9**

收集：　 

31　22　53　26　97　88　88’　59

(a) 第一趟分配个位

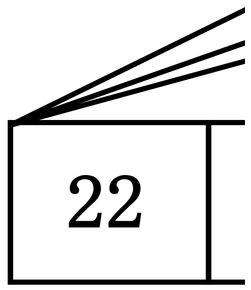
基于数组的LSD基数排序示例

十位：　0　1　2　3　4　5　6　7　8　9

第二趟：count　**0**　0　2　1　**1**　**2**　0　0　2　1

0　1　2　3　4　5　6　7　8　9

按 count 分配桶：　**0**　0　2　3　**4**　**6**　6　6　**8**　**9**



收集：　26　31　41　53　58　88　88’　97

最终排序结果：22　26　31　41　53　59　88　88’　97

(b) 第二趟分配十位

基于数组的LSD基数排序实现

```
template <typename Item>
void RadixSort_LSD(Item a[], int left, int right) {
    static Item aux[maxN];  int i, j, d, count[radix+1];
    if (right-left+1<=M) { InsertionSort(a, left, right);  return; }
    for (d = bytes-1; d >= 0; d--) {  // 从低位到高位
        for (j = 0; j <= radix; j++) count[j] = 0;
        for (i = left; i <= right; i++) count[getDigit(a[i], d)+1]++;
        for (j = 1;  j <= radix; j++) count[j] += count[j-1];
        for (i = left; i <= right; i++) aux[count[getDigit(a[i],d)]++] = a[i];
        for (i = left; i <= right; i++) a[i] = aux[i-left];
    }
}
```

基于链表的LSD基数排序示例

初始数组

97	53	88	59	26	41	88’	31	22

初始链表



第一趟分配

第一趟收集

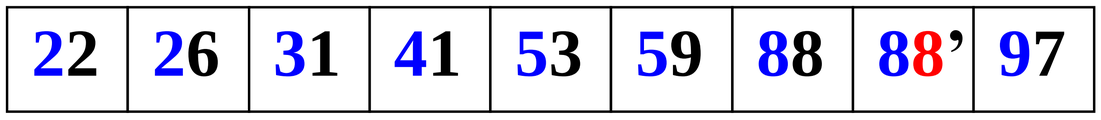
41	31	22	53	26	97	88	88’	59

基于链表的LSD基数排序示例



第二趟分配

第二趟收集
(最终结果)



基数排序性能分析

- 时间代价
  - 对数据进行d趟扫描，每趟需时间O(n+radix)，因此总的时间为O(d(n+radix))
  - 当n较大或d较小时，这种方法较为节省时间
- 空间代价：O(n)
- 稳定排序
- 对**整数和字符串**，基数排序的效率优于快速排序

内部排序方法的比较

- 基于“比较-交换”的排序时间复杂度理论下界Ω(nlogn)
  - 决策树证明
- 简单排序：插入排序、冒泡排序、选择排序
  - 需要O(1)的额外辅助空间
  - 平均时间复杂度均为$O(n^2)$
  - 只有选择排序是不稳定的
- 在元素个数较少时，插入排序性能好，比大多排序都快

内部排序方法的比较

- 高级排序：快速排序、堆排序、归并排序、希尔排序
- 快速排序：平均性能最好、最坏情况时间复杂度$O(n^2)$
- 堆排序：平均性能仅次于快速排序、仅需1个额外存储
- 归并排序：稳定的排序方法，没有最坏情况，这些高级排序中唯一稳定的排序方法
- 希尔排序：算法时间复杂度有待研究

内部排序方法的比较

- 基于“分配-收集”的排序：桶排序、基数排序
- 桶排序：待排元素均匀分配时，时间复杂度为O(n)
- 基数排序
  - 具有线性时间复杂度
  - 对整数与字符串有很高效率
  - 限于具体数据，效率与通用性不如基于“比较-交换”的排序

内部排序方法的比较

排序方法	平均时间	最坏时间	最佳时间	辅助空间	稳定性
插入排序	$O(n^2)$	$O(n^2)$	O(n)	O(1)	稳定
冒泡排序	$O(n^2)$	$O(n^2)$	O(n)(改进)	O(1)	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	不稳定
归并排序	O(n*logn)	O(n*logn)	O(n*logn)	O(n)	稳定
快速排序	O(n*logn)	$O(n^2)$	O(n*logn)	O(logn)	不稳定
堆排序	O(n*logn)	O(n*logn)	O(n*logn)	O(1)	不稳定
基数排序	O(d(n+rd))	O(d(n+rd))	O(d(n+rd))	O(n+rd)	稳定

排序算法的选择

- 当待排序记录数n较大时，若要求排序稳定，则采用归并排序
- 当待排序记录数n较大，关键字分布随机，而且不要求稳定时，可采用快速排序
- 当待排序记录数n较大，而且关键字会出现正、逆序情形，可采用堆排序(或归并排序)
- 当待排序记录数n较小，或者记录已接近有序时，可采用直接插入排序

练习

- 设有5000个无序的元素，希望用最快速度挑出其中前10个最大的元素。在快速排序、堆排序、归并排序、基数排序和希尔排序方法中，采用哪一种方法最好?为什么?
- 快速排序、归并排序、基数排序和希尔排序都是在排序结束之后才能确定数据元素的全部顺序，而无法知道排序过程中部分元素的有序性。但堆排序则每次输出一个最小(或最大)的元素，然后对堆进行调整，保证堆顶的元素是未排序元素中的最小(或最大)的。因此，选取前10个最大元素采用堆排序方法最好



- 堆排序、冒泡排序、插入排序、快速排序中哪一种可能会在最后一趟开始前，所有元素都不在其最终位置上？
- 插入排序



练习

- 对由n个元素组成的线性表进行快速排序时，所需进行的比较次数与这n个元素的初始排列有关
- 当n=7时，在最好情况下需进行多少次比较?
- 在最好情况下，每一趟快速排序后均能划分出左、右两个相等的区间。第一趟分出两个长度为3的子序列，共6次比较；第二趟分别对两个长度为3的子序列分出两个长度为1的子序列，共2*2=4次比较。因此，共需10次比较
- 当n=7时，在最坏情况下需进行多少次比较?
- 在最坏情况下，每趟用来划分的控制值总是定位于表的第一或最后一个位置，两个子序列一个长度为0，另一个仅是原表长减1，即比较次数为6+5+4+3+2+1=21次

练习

- 将n个正整数存放于数组中，试设计程序将所有的奇数移动到数组前半部分，并将所有的偶数移动到数组后半部分，要求尽可能少用额外存储空间且时间复杂度达到O(n)
- 采用快速排序中的一趟划分算法，从数组两端向中间检测，从右向左检查奇数，从左向右检查偶数，然后两者互换，如此重复执行，直到检测指针相遇

本章小结

- 基本排序方法
- 归并排序
- 快速排序
- 堆排序
- 希尔排序
- 基数排序

作业

- 设有n个元素随机存放于数组中，试编写程序返回数组中按关键码从小到大排序的第k个元素的值
- 设待排序的1000个关键码互不相同，其取值为1～10000的正整数，试编写程序以尽可能少的时间代价完成排序